

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Теория информации

Определение и исправления ошибок в сообщении.
Код Хемминга

Руководитель

подпись, дата

А.Н. Бочаров

инициалы, фамилия

Обучающийся БПИ24-02, 24151430

номер группы, зачетной книжки

подпись, дата

В.Т. Гордов

инициалы, фамилия

Красноярск 2026 г.

ЦЕЛЬ РАБОТЫ

Закрепление знаний по методам кодирования информации.

ЗАДАНИЯ

Построить код Хемминга для исправления одиночной ошибки и обнаружения двойной ошибки.

Код должен предусматривать возможность послышки n сообщений.

Варианты заданий в таблице 1:

Таблица 1

Вариант	Кол-во сообщений, n	Вариант	Кол-во сообщений, n
1	16	11	16384
2	32	12	32784
3	64	13	65536
4	128	14	128
5	256	15	256
6	512	16	512
7	1024	17	1024
8	2048	18	2048
9	4096	19	4096
10	8192	20	8192

1. Рассчитать параметры кода: n_u , n_k , n . Вариант задания определяется преподавателем.

2. Привести пример построения 10 кодовых сообщений.

3. Показать процедуру исправления ошибки в одной из позиций и обнаружения двойной ошибки.

4. Составить программу, кодирующую и декодирующую кодовую комбинацию с целью одиночной ошибки и обнаружения двойной ошибки.

ХОД РАБОТЫ

Для выполнения лабораторной работы было создано консольное приложение, написанное на языке Python. Исходный код программы представлен ниже.

Файл **header.py**

```
import math
import random

class HammingCoder:
    def __init__(self, num_messages):
        """Инициализация параметров кода Хемминга. 256 сообщений для 5 варианта."""
        self.n_i = int(math.log2(num_messages)) # количество информационных разрядов
        self.n_k = 0 # количество контрольных разрядов
        # формула для вычисления
        while (2 ** self.n_k) < (self.n_i + self.n_k + 1):
            self.n_k += 1
        self.n = self.n_i + self.n_k # длина классического кода Хемминга (разрядность кода)
        self.n_ext = self.n + 1 # длина расширенного кода Хемминга

    def generate_data_string(self):
        """Генерация случайной последовательности данных."""
        return ''.join(random.choice("01") for _ in range(self.n_i))

    def encode(self, data_string):
        """Кодирование информационной строки расширенным кодом Хемминга."""
        # Массив для формирования кода (индексация с 1 для удобства степеней двойки)
        code = [0] * (self.n + 1)
        # Расставляем информационные биты на позиции, не являющиеся степенями 2
        j = 0
        for i in range(1, self.n + 1):
            if (i & (i - 1)) != 0:
                code[i] = int(data_string[j])
                j += 1
        # Расставляем контрольные биты на остальные позиции
        for i in range(self.n_k):
            pos = 2 ** i
            parity = 0
            for k in range(1, self.n + 1):
                # Если в позиции k бит, проверяемый текущим контрольным разрядом, равен 1
                if (k & pos) != 0:
                    parity ^= code[k] # XOR сумма
            code[pos] = parity
        # Вычисляем общий бит чётности для расширенного кода (сумма всех бит)
        parity_bit = sum(code[1:]) % 2
        # Формируем итоговую строку (исключаем нулевой индекс)
        result = ''.join(map(str, code[1:])) + str(parity_bit)
        return result
```

```

def decode(self, code_str):
    """Декодирование кодовой строки с проверкой на наличие одиночных и двойных
    ошибок."""
    # Преобразуем строку в список чисел (первый элемент фиктивный для 1-индексации)
    code = [0] + [int(x) for x in code_str[:-1]]
    parity_bit = int(code_str[-1]) # бит чётности
    status = ""
    corrected_code = code.copy()
    corrected_overall = parity_bit
    # Проверка общей чётности
    current_overall_parity = (sum(code[1:]) + parity_bit) % 2
    # Вычисление синдрома
    syndrome = 0
    for i in range(self.n_k):
        pos = 2 ** i
        parity = 0
        for k in range(1, self.n + 1):
            if (k & pos) != 0:
                parity ^= code[k]
        if parity != 0:
            syndrome += pos # Накапливаем номер ошибочного бита
    # Принятие решения по теореме Хемминга
    # Ошибка исправляется инверсией бита
    if current_overall_parity == 0:
        if syndrome == 0:
            status = "Ошибок нет."
        else:
            status = f"Двойная ошибка (синдром = {syndrome}). Исправление невозможно."
    else:
        if syndrome == 0:
            status = "Одиночная ошибка в общем бите чётности. Исправление выполнено."
            corrected_overall ^= 1
        else:
            status = f"Одиночная ошибка в позиции {syndrome}. Исправление выполнено."
            corrected_code[syndrome] ^= 1
    # Извлечение информационной части из исправленного кода
    data = ""
    for i in range(1, self.n + 1):
        if (i & (i - 1)) != 0:
            data += str(corrected_code[i])
    final_code_str = "".join(map(str, corrected_code[1:])) + str(corrected_overall)
    return status, final_code_str, data

```

Файл **main.py**

```

from header import HammingCoder

```

```

def simulate_error(code_str, positions):
    """Внесение ошибок в строку."""

```

```

code_list = list(code_str)
for pos in positions:
    idx = pos - 1
    if 0 <= idx < len(code_list):
        code_list[idx] = '1' if code_list[idx] == '0' else '0'
return "".join(code_list)

def main():
    # 5 вариант: 256 сообщений
    coder = HammingCoder(num_messages=256)

    while True:
        print("\n--- ЛАБОРАТОРНАЯ РАБОТА № 4 ---")
        print("1. Задание 1: Рассчёт параметров кода")
        print("2. Задание 2: Пример построения 10 кодовых сообщений")
        print("3. Задания 3-4: Демонстрация исправления и обнаружения ошибок")
        print("0. Выход")

        choice = input("Выбор: ")

        if choice == "1":
            print("\n" + "="*40)
            print("ПАРАМЕТРЫ КОРРЕКТИРУЮЩЕГО КОДА")
            print("="*40)
            print(f"Количество сообщений (N): {256}")
            print(f"Информационных разрядов (n_и): {coder.n_i}")
            print(f"Контрольных разрядов (n_к): {coder.n_k}")
            print(f"Общая длина классического кода Хемминга (n): {coder.n}")
            print(f"Длина расширенного кода: {coder.n_ext}")

        elif choice == "2":
            print("\n" + "="*40)
            print("ПОСТРОЕНИЕ 10 СЛУЧАЙНЫХ СООБЩЕНИЙ")
            print("="*40)
            print(f"{'№':<3} | {'Инфо-данные':<12} | {'Расширенный код Хемминга'}")
            print("-" * 50)
            for i in range(1, 11):
                # Генерируем случайную 8-битную строку
                data = coder.generate_data_string()
                encoded = coder.encode(data)
                print(f"{i:<3} | {data:<12} | {encoded}")

        elif choice == "3":
            print("\n" + "="*40)
            print("СИМУЛЯТОР КАНАЛА С ПОМЕХАМИ")
            print("="*40)
            data = coder.generate_data_string()
            encoded = coder.encode(data)
            print(f"Сгенерированные данные: {data}")
            print(f"Отправленный в канал код: {encoded}")

```

```

    err_input = input("\nУкажите до двух позиций ошибок при прохождении через канал
(например, 5 или 3,7): ").strip()
    err_positions = [int(x.strip()) for x in err_input.split(",") if x.strip().isdigit()]

    received = simulate_error(encoded, err_positions)
    print(f"\nПолученный из канала код (с ошибками): {received}")

    status, corrected, decoded_data = coder.decode(received)
    print(f"\nСтатус декодирования: {status}")
    print(f"\nИсправленный код: {corrected}")
    print(f"\nИтоговая последовательность данных: {decoded_data}")

elif choice == "0":
    break
else:
    print("Неверный ввод.")

if __name__ == "__main__":
    main()

```

В коде представленной программы был реализован класс **HammingCoder**, предназначенный для кодирования и декодирования информации методом Хемминга.

Взаимодействие с функциональными методами происходит с помощью консольного меню (Рисунок 1).

```

--- ЛАБОРАТОРНАЯ РАБОТА № 4 ---
1. Задание 1: Рассчёт параметров кода
2. Задание 2: Пример построения 10 кодовых сообщений
3. Задания 3-4: Демонстрация исправления и обнаружения ошибок
0. Выход
Выбор: █

```

Рисунок 1 — Меню программы

Задание 1

При выборе 1 пункта программа вычисляет и выводит параметры (Рисунок 2), среди которых:

- Количество информационных разрядов кода $n_u = \log_2 N$;
- Количество контрольных разрядов n_k : $2^{n_k} \geq n_u + n_k + 1$;
- Общая длина кодового сообщения $n = n_u + n_k$.

```

=====
ПАРАМЕТРЫ КОРРЕКТИРУЮЩЕГО КОДА
=====
Количество сообщений (N): 256
Информационных разрядов (n_и): 8
Контрольных разрядов (n_к): 4
Общая длина классического кода Хемминга (n): 12
Длина расширенного кода: 13

```

Рисунок 2 — Вывод параметров кода Хемминга

Задание 2

При выборе 2 пункта программа последовательно генерирует 10 случайных 8-ми разрядных комбинаций, после чего строит расширенный код Хемминга для каждого из них. Результаты представлены на Рисунке 3.

```

=====
ПОСТРОЕНИЕ 10 СЛУЧАЙНЫХ СООБЩЕНИЙ
=====

```

№	Инфо-данные	Расширенный код Хемминга
1	01111110	0001111111100
2	00100111	1100010101111
3	00000000	0000000000000
4	01010011	1001101000110
5	11001111	0110100011111
6	00111000	0000011110000
7	01111000	1001111110001
8	11001010	0011100010101
9	10011000	1011001110000
10	11001000	1111100110001

Рисунок 3 — Расширенный код Хемминга для 10 сообщений

Задания 3-4

При выборе 3 пункта программа генерирует последовательность данных и строит расширенный код Хемминга. Затем от пользователя требуется указать позиции кодовой строки, на которых должны быть смоделированы ошибки.

После ввода данных пользователем символы на выбранных позициях инвертируются, симулируя возникновение ошибок при прохождении кода через канал связи. Далее предпринимается попытка обнаружить и исправить ошибки (в зависимости от их количества) и декодировать кодовую последовательность в информационную строку. Демонстрация работы представлена на Рисунках 4-6.


```

=====
СИМУЛЯТОР КАНАЛА С ПОМЕХАМИ
=====
Сгенерированные данные: 10010101
Отправленный в канал код: 0110001001011

Укажите до двух позиций ошибок при прохождении через канал (например, 5 или 3,7): 1

Полученный из канала код (с ошибками): 1110001001011

Статус декодирования: Одиночная ошибка в позиции 1. Исправление выполнено.
Исправленный код: 0110001001011
Итоговая последовательность данных: 10010101

```

Рисунок 4 — Симуляция одной ошибки в коде Хемминга

```

=====
СИМУЛЯТОР КАНАЛА С ПОМЕХАМИ
=====
Сгенерированные данные: 11000100
Отправленный в канал код: 0011100101001

Укажите до двух позиций ошибок при прохождении через канал (например, 5 или 3,7): 3,5

Полученный из канала код (с ошибками): 0001000101001

Статус декодирования: Двойная ошибка (синдром=6). Исправление невозможно.
Исправленный код: 0001000101001
Итоговая последовательность данных: 00000100

```

Рисунок 5 — Симуляция двух ошибок в коде Хемминга

```

=====
СИМУЛЯТОР КАНАЛА С ПОМЕХАМИ
=====
Сгенерированные данные: 00111000
Отправленный в канал код: 0000011110000

Укажите до двух позиций ошибок при прохождении через канал (например, 5 или 3,7):

Полученный из канала код (с ошибками): 0000011110000

Статус декодирования: Ошибок нет.
Исправленный код: 0000011110000
Итоговая последовательность данных: 00111000

```

Рисунок 6 — Кодирование и декодирование кода Хемминга без ошибок

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое корректирующий код?

Код, позволяющий обнаруживать и исправлять ошибки, возникающие при передаче информации через канал связи с помехами.

2. Как обнаруживаются ошибки при передаче информации при помощи корректирующего кода?

Для передачи информации используются не все возможные кодовые комбинации, а только их часть — разрешённые коды, остальные являются запрещёнными. Если разрешённая комбинация искажается, то она превращается в запрещённую, что указывает на наличие ошибки.

3. Как рассчитать параметры корректирующего кода?

- Количество информационных разрядов кода $n_u = \log_2 N$;
- Количество контрольных разрядов n_k : $2^{n_k} \geq n_u + n_k + 1$;
- Общая длина кодового сообщения $n = n_u + n_k$.

4. Как определить кодовое расстояние?

Кодовое расстояние d это количество разрядов, в которых две комбинации отличаются друг от друга. Для его определения нужно сложить кодовые комбинации по модулю 2 и подсчитать количество единиц в полученной сумме.

5. Как определить минимальное кодовое расстояние?

d_{min} находится путём попарного сравнения всех разрешённых кодовых комбинаций данного кода и выбора наименьшего из полученных расстояний.

Иначе: $d_{min} = g_0 + g_u + 1$, где g_0 и g_u это число обнаруживаемых и исправляемых ошибок соответственно.

6. Как строится код Хемминга?

1. Вычисляется количество контрольных разрядов n_k .
2. Контрольные разряды размещаются на позициях, номера которых есть степени 2 (1, 2, 4, ...).
3. Информационные разряды размещаются на остальных позициях.
4. Значения контрольных разрядов вычисляются с помощью схем проверки на чётность. Сумма единиц в каждой проверяемой группе должна быть чётной.

7. Как проводится процедура декодирования кода Хемминга?

1. На стороне приёмника заново проверяется чётность всех групп контрольных разрядов.
2. Результаты проверок записываются в вектор синдрома (0 или 1).
3. Синдром равен нулю — нет ошибок.
4. Синдром ненулевой — его десятичное значение указывает на позицию кода с ошибкой.

8. Что такое пропускная способность канала связи?

Пропускная способность C — это максимально возможное количество информации, которое канал способен передать в единицу времени: $C = \max_{P(X)} [H(X) - H(X/Y)]$. Определяется как максимум взаимной информации между входом и выходом канала по всем возможным распределениям вероятностей на входе.

9. В чем суть теоремы Шеннона для дискретного канала с помехами?

При наличии дискретного канала с помехами возможно с бесконечно малой вероятностью ошибки передавать информацию, если скорость передачи сообщений источника не превышает пропускную способность C канала. Это достигается за счет применения помехоустойчивого кодирования.

10. Как описать двоичный канал?

Двоичный канал описывается входным алфавитом $X = \{x_1, x_2\}$ и выходным $Y = \{y_1, y_2\}$, а также матрицей условных вероятностей переходов.

11. Как определить пропускную способность двоичного канала?

$C = v_x \cdot [1 + (1-p)\log_2(1-p) + p\log_2 p]$. p – вероятность ошибки.

12. Что представляет собой вектор ошибок при обнаружении и исправлении ошибок в сообщении?

Вектор ошибок, он же «синдром», это двоичная последовательность, которая формируется на стороне приёмника на основании правил проверки. Значение синдрома в десятичном виде указывает либо на позицию ошибки в коде Хемминга, либо на её наличие, в зависимости от количества ошибок.

ВЫВОДЫ

В ходе выполнения лабораторной работы были закреплены теоретические знания по методам кодирования информации. Были получены практические навыки по кодированию и декодированию информационных последовательностей с помощью расширенного кода Хемминга.

Исходя из полученных результатов, можно сделать следующие выводы:

Определение параметров кода: Для кодирования 256 возможных сообщений требуется информационный блок $n_u = 8$ бит. Для обеспечения возможности исправления одиночной и обнаружения двойной ошибки требуется добавить $n_k = 4$ контрольных разряда для составления классического кода Хемминга. Для расширенного кода Хемминга дополнительно добавляется бит чётности, в следствии чего длина расширенного кода составляет 13 бит.

Исправление одиночной ошибки: При возникновении одиночной ошибки в коде нарушается общая чётность кодовой комбинации, что можно отследить с помощью бита чётности. Ненулевой синдром, вычисляемый по искажённому коду, позволяет точно определить позицию ошибки и инвертировать повреждённый бит.

Обнаружение двойной ошибки: При искажении двух битов в кодовой последовательности общая чётность сохраняется, но вычисляемый синдром не равен нулю. В такой ситуации возможно точно диагностировать факт двойной ошибки, но без возможности исправления.